

Offline Deployment Guide

Automation Deployment

1. Copy file bag-doorprize-deploy.tar.gz to folder /home/sysadmin/bagi-hoki-main/YYYY-MM-DD/bag-doorprize-deploy.tar.gz.

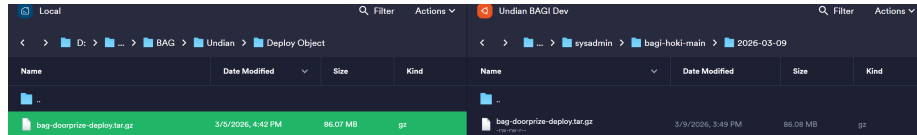


Figure 1: Copy file bag-doorprize-deploy.tar.gz to server

2. Make sure you copy file deploy.sh to /home/sysadmin/ or /home/root/ (Based on project placed).

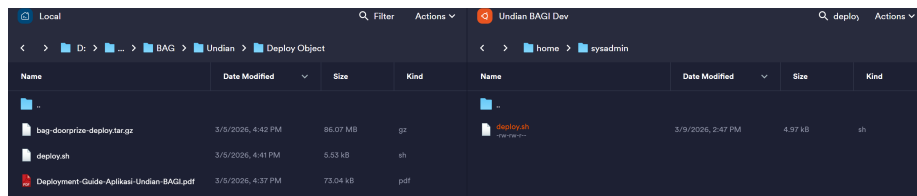


Figure 2: Copy deploy.sh to server

3. Then run this script to make file deploy.sh executable `sudo chmod +x deploy.sh`

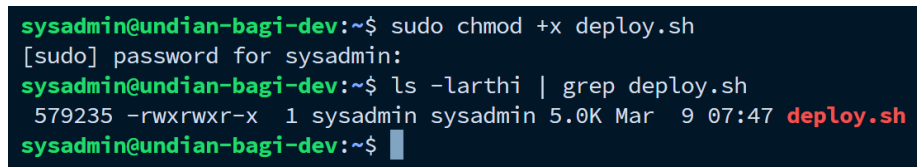


Figure 3: Give permission executable file for deploy.sh

4. Run file deploy.sh by this script `sudo bash deploy.sh`
5. Deployment successfull

Notes:

If you have implemented automation deploy, preparation below will be optional.

```
sysadmin@undian-bagi-dev:~$ sudo bash deploy.sh

[INFO] Starting deployment: bagi-hoki
[INFO] Time: Mon Mar 9 09:48:59 UTC 2026

[OK] Deployment archive found.

[INFO] Step 1/6 – Stopping services...
laravel-octane: stopped
laravel-worker:laravel-worker_00: stopped
laravel-worker:laravel-worker_01: stopped
laravel-worker:laravel-worker_02: stopped
laravel-worker:laravel-worker_03: stopped
laravel-worker:laravel-worker_04: stopped
laravel-worker:laravel-worker_05: stopped
laravel-worker:laravel-worker_06: stopped
laravel-worker:laravel-worker_07: stopped
[OK] Services stopped.

[INFO] Step 2/6 – Creating lightweight backup...
[OK] Backup created:
/home/sysadmin/bagi-hoki-backup/backup-20260309-094859.tar.gz

[INFO] Step 3/6 – Cleaning old deployment...
[OK] Old files removed.

[INFO] Step 4/6 – Extracting new release...
[OK] Extraction completed.
```

Figure 4: Run bash deploy.sh

```
>>> Deployment complete! App is ready.  
[OK]   Application deployment completed.  
  
[INFO] Step 6/6 – Restarting services...  
laravel-octane: started  
laravel-worker:laravel-worker_00: started  
laravel-worker:laravel-worker_01: started  
laravel-worker:laravel-worker_02: started  
laravel-worker:laravel-worker_03: started  
laravel-worker:laravel-worker_04: started  
laravel-worker:laravel-worker_05: started  
laravel-worker:laravel-worker_06: started  
laravel-worker:laravel-worker_07: started  
[OK]   Services restarted.  
  
[INFO] Checking nginx status...  
[OK]   Nginx is running.  
  
✓ DEPLOYMENT SUCCESSFUL  
Time: Mon Mar 9 09:49:07 UTC 2026
```

Figure 5: Successfully running deployment

Manual Deployment

This guide explains how to deploy the application to a server with **no internet connection**.

1. Local Preparation (With Internet)

Run these steps on your development machine where you have internet access.

1. Open your terminal in the project root.
2. Execute the following command to build and package the app: `bash make package`
3. This will:
 - Install production dependencies (`vendor/`).
 - Build frontend assets (`public/build/`).
 - Compact everything into a file named `bag-doorprize-deploy.tar.gz`.

2. Server Setup (Offline)

Move the generated `bag-doorprize-deploy.tar.gz` to your target server using a USB drive or local network.

1. Extract the package: `bash tar -xzf bag-doorprize-deploy.tar.gz -C /path/to/webroot`
2. Navigate to the project directory: `bash cd /path/to/webroot`
3. Create and configure your `.env` file manually: `bash cp .env.example .env nano .env # Adjust DB_HOST, DB_PASSWORD, etc. # Set OCTANE_SERVER=frankenphp`
4. Run the deployment script: `bash make deploy`
5. Setup the automated task scheduler (Crontab): `bash make cron`

3. Running the Application (Octane + FrankenPHP)

This application is designed to run with **Laravel Octane** and **FrankenPHP**.

To start the application server:

```
make octane
```

The server will start on port 8000 by default (as defined in the Makefile).

4. Setup User Management & Permissions

Create Super Admin User

Create a super admin user using this command:

```
php artisan shield:super-admin
```

This will prompt you for:

- Name
- Email
- Password

The command will create a user with the `super_admin` role and all permissions.

Install Shield Filament Generator (Optional - For Development)

If you need to regenerate Shield resources or customize permissions, install the Shield generator:

1. **Install the package** (requires internet):

```
composer require bezhansalleh/filament-shield --dev
```

2. **Publish Shield resources:**

```
php artisan vendor:publish --tag="filament-shield-config"
```

3. **Generate Shield resources** (roles, permissions UI):

```
php artisan shield:install
```

This will:

- Create migrations for roles and permissions
- Generate Shield resources (RoleResource, etc.)
- Set up permission policies

4. **Generate permissions for existing resources:**

```
php artisan shield:generate --all
```

This scans all Filament resources and creates permissions for them.

5. **Create custom roles** (if needed): `bash` `php artisan shield:create-role`

Shield Commands Reference

Command	Description
<code>php artisan shield:super-admin</code>	Create a super admin user
<code>php artisan shield:install</code>	Install Shield resources
<code>php artisan shield:generate --all</code>	Generate permissions for all resources
<code>php artisan shield:generate --resource=CustomerResource</code>	Generate permissions for specific resource
<code>php artisan shield:create-role</code>	Create a new role interactively
<code>php artisan shield:publish</code>	Publish Shield views and config

Managing Permissions

After installation, you can manage roles and permissions through the Filament admin panel:

1. Navigate to `/admin/shield/roles`
2. Create/edit roles
3. Assign permissions to roles
4. Assign roles to users

Notes

- **PHP & Web Server:** Ensure the server already has PHP 8.2+, a database (PostgreSQL/MySQL), and required extensions. FrankenPHP handles web requests directly, so a separate Nginx/Apache is optional.
- **Binary Status:** The `frankenphp` binary is bundled within the deployment package. The `make deploy` command automatically sets the correct execution permissions for it.